

Note su Program (4).cs e Program (5).cs

Program (4).cs – Implementazione di Metodi LINQ

Concetti Principali

Questo file implementa da zero i **metodi LINQ standard** di .NET, mostrando come funzionano internamente.

Parte 1: IEnumerable e IEnumerator (Commentata)

IEnumerable: Rappresenta una collezione iterabile.

IEnumerator: È l'oggetto che esegue effettivamente l'iterazione.

```
IEnumerable<char> enu = s;  
IEnumerator<char> ator = enu.GetEnumerator();  
while (ator.MoveNext())  
    Console.Write($"{ator.Current}.");
```

Come funziona:

1. `GetEnumerator()` – Crea un iteratore
2. `MoveNext()` – Avanza al prossimo elemento (true/false)
3. `Current` – Accede all'elemento corrente
4. `Dispose()` – Libera le risorse

Parte 2: Lettura File con IEnumerable (Commentata)

La funzione `FileLines()` usa `yield return` per leggere un file riga per riga **lazy** (solo quando richiesto):

```
IEnumerable<string> FileLines (string filename) {  
    string line = "";  
    using (var stm = File.Open (filename, FileMode.Open))  
    using (var sr = new StreamReader (stm)) {  
        for (; ; ) {  
            int n = sr.Read();  
            if (n == -1) break;           // Fine file  
            if (n == '\r') continue;     // Ignora \r  
            if (n == '\n') {             // Nuova riga  
                yield return line;  
                line = "";  
            }  
        }  
    }  
}
```

```

    continue;
}
line += (char)n;
}
}
if (line != "") yield return line;    // Ultima riga se non vuota
}

```

Vantaggio: Il file viene letto gradualmente, non tutto in memoria.

Parte 3: Implementazione dei Metodi LINQ (ATTIVA)

1. Where – Filtrare Elementi

```

static public IEnumerable<T> Where<T> (this IEnumerable<T> seq, Func<T,
bool> predicate) {
    foreach (var elem in seq) {
        if (predicate(elem))
            yield return elem;
    }
}

```

Uso: `a.Where(n => n % 2 == 0)` → Numeri pari

Output: 2 4 6 8 10

Cosa fa: Itera sulla sequenza e restituisce solo elementi che soddisfano la condizione.

2. Select – Trasformare Elementi

```

static public IEnumerable<U> Select<T, U> (this IEnumerable<T> seq,
Func<T, U> transform) {
    foreach (var elem in seq) {
        yield return transform(elem);
    }
}

```

Uso: `a.Select(n => n*n)` → Quadrati

Output: 1 4 9 16 25 36 49 64 81 100

Cosa fa: Applica una funzione a ogni elemento e restituisce i risultati trasformati.

3. Min – Trovare il Minimo

```
static public T Min<T> (this IEnumerable<T> seq) where T : IComparable<T>
{
    T min = default;
    bool first = true;
    foreach (var elem in seq) {
        if (first) {
            min = elem;
            first = false;
        } else if (elem.CompareTo(min) < 0)
            min = elem;
    }
    if (first) throw new Exception("Sequence empty");
    return min;
}
```

Uso: `a.Min()` → 1

Cosa fa: Compara ogni elemento e mantiene il più piccolo.

4. Take – Prendi i Primi N Elementi

```
static IEnumerable<T> Take<T> (this IEnumerable<T> seq, int count) {
    foreach (var elem in seq) {
        if (--count < 0) break;
        yield return elem;
    }
}
```

Uso: `a.Take(3)` → 1 2 3

Come funziona:

- `--count` decrementa il contatore prima della comparazione
- Quando `count < 0`, esce dal ciclo

5. Skip – Salta i Primi N Elementi

```
static IEnumerable<T> Skip<T> (this IEnumerable<T> seq, int count) {
    foreach (var elem in seq) {
        if (--count >= 0) continue;
        yield return elem;
    }
}
```

```
}  
}
```

Uso: `a.Skip(3)` → 4 5 6 7 8 9 10

Come funziona: Decrementa e salta finché `count >= 0`, poi restituisce il resto.

6. SkipLast – Salta gli Ultimi N Elementi

```
static IEnumerable<T> SkipLast<T> (this IEnumerable<T> seq, int count) {  
    var queue = new Queue<T>();  
    foreach (var elem in seq) {  
        queue.Enqueue(elem);  
        if (queue.Count > count)  
            yield return queue.Dequeue();  
    }  
}
```

Uso: `a.SkipLast(0)` → 1 2 3 4 5 6 7 8 9 10

Come funziona: Usa una Queue per "ritardare" di N elementi l'output.

- Quando `Count > N`, estrae (`Dequeue`) il primo elemento
- Così gli ultimi N rimangono nella coda senza uscire

7. Zip – Accoppia Elementi di Due Sequenze

```
static public IEnumerable<(T, U)> Zip<T, U> (this IEnumerable<T> seq1,  
IEnumerable<U> seq2) {  
    var (enu1, enu2) = (seq1.GetEnumerator(), seq2.GetEnumerator());  
    var (next1, next2) = (enu1.MoveNext(), enu2.MoveNext());  
    while (next1 && next2) {  
        yield return (enu1.Current, enu2.Current);  
        next1 = enu1.MoveNext();  
        next2 = enu2.MoveNext();  
    }  
}
```

Uso: `a.Zip(b)` dove `a = {1,2,3,4,5...}` e `b = {"one","two","three","four"}`

Output: (1, one) (2, two) (3, three) (4, four)

Come funziona: Sincronizza due iteratori e accoppia elementi paralleli.

8. MergeSort – Merge di Due Sequenze Ordinate

```
static public IEnumerable<T> MergeSort<T> (this IEnumerable<T> seq1,
IEnumerable<T> seq2)
    where T: IComparable<T> {
    var (enu1, enu2) = (seq1.GetEnumerator(), seq2.GetEnumerator());
    var (next1, next2) = (enu1.MoveNext(), enu2.MoveNext());
    while (next1 || next2) {
        bool pickFirst = !next2 || (next1 && enu1.Current.CompareTo(enu2.Current) < 0);
        if (pickFirst) {
            yield return enu1.Current;
            next1 = enu1.MoveNext();
        } else {
            yield return enu2.Current;
            next2 = enu2.MoveNext();
        }
    }
}
```

Uso:

- `c = {1, 3, 5, 7, 9, 10, 11, 12, 13}`
- `d = {2, 4, 6, 8, 10, 20}`
- `c.MergeSort(d) → 1 2 3 4 5 6 7 8 9 10 10 11 12 13 20`

Come funziona:

1. Compara il primo elemento di seq1 con seq2
2. Estrae il più piccolo
3. Continua finché una (o entrambe) non sono esaurite
4. `!next2 || (next1 && ...)` → prende da seq1 se seq2 è finita oppure se `seq1.Current < seq2.Current`

Program (5).cs – Ricerca Anagrammi con Confronto Performance

Obiettivo

Trovare tutti i **gruppi di anagrammi** in una lista di parole (ad es. "amor", "mora", "roma").

Strategia Comune

Un anagramma è un gruppo di parole che contengono le **stesse lettere in ordine diverso**.

Idea: Se ordino alfabeticamente le lettere di ogni parola, tutte gli anagrammi produrranno la **stessa chiave**.

Esempio:

- "amor" → ordinato = "amor"
- "mora" → ordinato = "amor"
- "roma" → ordinato = "amor"
- Chiave: **"amor"**

Opzione 1: Dictionary Manuale con Loop

```
var anagram = new Dictionary<string, List<string>>();
foreach (var word in File.ReadAllLines("C:/Academy/A14/esA14/words.txt")) {
    string key = new string(word.Order().ToArray());

    if (!anagram.TryGetValue(key, out var list))
        anagram.Add(key, list = new());

    list.Add(word);
}
```

Flusso:

1. `word.Order()` → Ordina i caratteri alfabeticamente
2. `ToArray()` → Converte in array di char
3. `new string(...)` → Crea la stringa chiave
4. `TryGetValue()` → Evita eccezioni: se chiave esiste, recupera la lista
5. Se non esiste, crea una nuova lista e l'aggiunge

Vantaggi: Semplice, facilmente controllabile

Performance: Media

Opzione 2: GroupBy (LINQ)

```
var words2 = File.ReadAllLines("C:/Academy/A14/esA14/words.txt");
var anagram2 = words2.GroupBy(a => new string(a.Order().ToArray()))
    .Where(a => a.Count() >= 2)
    .OrderByDescending(a => a.Count());
```

Flusso:

1. `GroupBy()` → Raggruppa parole per chiave (stesse lettere ordinate)
2. `Where(a => a.Count() >= 2)` → Filtra solo gruppi con 2+ elementi
3. `OrderByDescending()` → Ordina dal gruppo più grande al più piccolo

Vantaggi: Codice compatto e leggibile

Performance: Migliore (LINQ è ottimizzato)

Opzione 3: Hashing con Numeri Primi

```
int[] Primes = { 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, ... };

int ComputeKey(string word) {
    int hash = 1;
    foreach (var ch in word.Where(a => a is >= 'A' and <= 'Z'))
        checked { hash *= Primes[ch - 'A']; }
    return hash;
}

var anagram3 = new Dictionary<int, List<string>>();
foreach (var word in File.ReadAllLines("C:/Academy/A14/esA14/words.txt")) {
    int key = ComputeKey(word);
    if (!anagram3.TryGetValue(key, out var list))
        anagram3.Add(key, list = new());
    list.Add(word);
}
```

Come funziona:

- Assegna un numero primo a ogni lettera (A=2, B=3, C=5, ...)
- Moltiplica i numeri primi delle lettere presenti
- Due parole sono anagrammi se il prodotto è uguale

Esempio:

- "amor" = $P(a) \times P(m) \times P(o) \times P(r) = 2 \times 13 \times 15 \times 18 = 7020$
- "mora" = $P(m) \times P(o) \times P(r) \times P(a) = 13 \times 15 \times 18 \times 2 = 7020$
- "roma" = $P(r) \times P(o) \times P(m) \times P(a) = 18 \times 15 \times 13 \times 2 = 7020$

Vantaggi: Molto veloce (usa interi invece di stringhe)

Svantaggi: Può causare **overflow** se le parole sono lunghe (per questo il `checked`)

Confronto Performance

Il programma misura i tempi con `Stopwatch`:

```
Stopwatch sw = Stopwatch.StartNew();  
// ... codice ...  
sw.Stop();  
Console.WriteLine($"Time: {sw.Elapsed.TotalMilliseconds} [ms]");
```

Risultati Tipici:

- **Opzione 1** (Dictionary manuale): ~50-100 ms
- **Opzione 2** (GroupBy): ~30-60 ms ← Migliore
- **Opzione 3** (Prime Hash): ~20-40 ms ← Molto veloce, ma rischioso con `checked`

Output

Il programma scrive il risultato in un file di testo formattato:

NUM	CHIAVE	ANAGRAMMI
15	aelopt	palate, petal, plate, pleat, sepal, splat...
7	aeorst	roasts, sorest, stores, taeros...
...		

- **NUM**: Quante parole hanno questa stessa combinazione di lettere
- **CHIAVE**: Le lettere ordinate
- **ANAGRAMMI**: Tutte le parole che corrispondono

Concetti Chiave da Ricordare

Concetto	Spiegazione
yield return	Restituisce elementi uno alla volta (lazy evaluation)
IEnumerable	Interfaccia per collezioni iterabili
LINQ	

	Sintassi per query su sequenze (Where, Select, GroupBy, etc.)
lambda	Funzione anonima (es. <pre>n => n % 2 == 0</pre>)
Anagramma	Parole con le stesse lettere in ordine diverso
Hashing	Convertire dati in un valore univoco (chiave)
Performance	Opzione 2 (GroupBy) è il miglior equilibrio qualità/velocità

Takeaways Pratici

1. **Quando usare Where/Select:** Quando devi filtrare o trasformare dati
2. **Quando usare GroupBy:** Quando devi raggruppare dati per categorie
3. **Quando usare Zip:** Quando devi sincronizzare due sequenze
4. **Quando usare MergeSort:** Quando devi unire due liste già ordinate
5. **Lazy Evaluation:** `yield return` è utile per file grandi (non caricare tutto in memoria)